

變數可見範圍與生命週期

- **12-1:宣告區域變數與類別變數**

區域變數.....	12-1
區域變數的宣告位置及有效範圍.....	12-2
什麼是類別變數.....	12-3
區域變數與類別變數.....	12-4

- **12-2:變數存取修飾詞-public internal private**

存取修飾詞一覽.....	12-5
常用的存取修飾.....	12-6

- **12-3:static 關鍵字**

static靜態成員(Static Members).....	12-7
使用static.....	12-8
static的特性.....	12-9

- **12-4:靜態變數與執行個體變數**

何謂靜態變數.....	12-10
何謂執行個體變數.....	12-11

Module 12

區域變數

- 區域變數(**local variables**)是在方法函式內宣告的變數(或方法的引數)，區域變數一離開方法，其生命週期即馬上結束。
 - **Block level**：區塊變數，區段中的變數。
 - `if( ... )`
`{`
`int count = 0; count = 1;     //count為區塊變數`
`}`
 - **Method level**：方法變數，方法中的變數。
 - `void DukeKnow()`
`{`
`string Duke = “浪漫Duke地下有知”;     //Duke為方法變數`
`}`

區域變數的宣告位置及有效範圍

	區段	方法
宣告語法	如宣告變數	
存取控制	無	
可見範圍	區段中	方法中

```
void MyVoid1()
{
    //此處無法存取a及x變數
    int a = 1; //方法
    if (true)
    {
        int x = 1; //區塊
        a = x;
    }
    //此處無法存取x變數
}

void MyVoid2()
{
    //此處無法存取a及x變數
}
```

什麼是類別變數

- 一般在類別中的變數，當個別建立類別物件，該變數只限於個別物件所使用。
- 宣告方式與變數規則相同。
- 使用存取修飾詞控制存取權限。
- 範例：

```
public class MyClass
{
    public int num = 10;
    internal string foo = "Foo";
    protected int m_num;
}
```

區域變數與類別變數

	區段	方法	類別
宣告語法	如宣告變數		
存取控制	無		有
可見範圍	區段中	方法中	類別中

```
public class Program
{
    int b = 1; //類別變數
    void MyVoid1()
    {
        //此處無法存取a及x變數
        int a = 1; b = a; //方法
        if (true)
        {
            int x = 1; //區塊
            a = b = x;
        }
        //此處無法存取x變數
    }
    void MyVoid2()
    {
        b = 2;
        //此處無法存取a及x變數
    }
}
```

存取修飾詞一覽

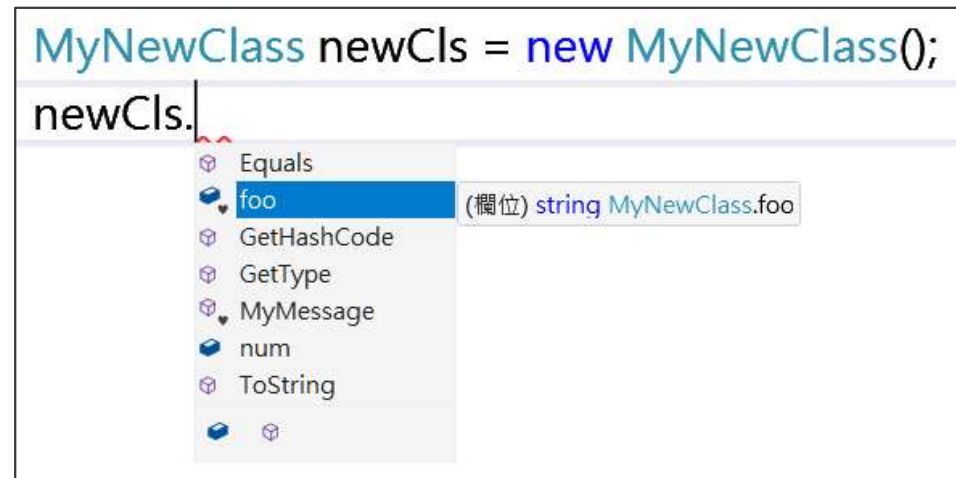
存取修飾詞/屬性	說明
private	只有類別本身可以使用
protected	可被類別本身及子類別使用
public	任何地方都可使用
internal	同一組件(專案)中所有的類別都可使用
protected internal	可被同一組件的類別、類別本身及子類別使用

- 類別（ **class** ）：若未明確指定存取修飾詞，預設為 **internal**
- 類別中的成員（ 屬性、方法等 ）：若未指定存取修飾詞，預設為 **private**
- 介面（ **interface** ）中的成員：若未指定存取修飾詞，預設為 **public**

常用的存取修飾詞

- 實例中較常使用的存取修飾詞：
 - **public, internal, private**
- 範例：

```
public class MyNewClass
{
    public int num = 10;
    internal string foo = "Foo";
    private int m_num;
    internal void MyMessage()
    {
        MessageBox.Show("存取修飾詞");
    }
}
```



static 靜態成員 (Static Members)

- 使用static關鍵字宣告「static靜態成員」(屬性、方法、或類別)。
- 靜態成員不需建立物件實體(new)，即可直接透過類別名稱呼叫。
- 在記憶體中僅會存在一份共用實例，所有物件都可共用該成員。
- 靜態成員適合用於全域共用資料或工具類別(Utility Class)。

使用static

- 在類別、介面和結構中，**static**可加入至以下：
 - 欄位(變數)、方法、屬性、事件。
 - **static**關鍵字套用至類別，該類別的所有成員都必須**static**
- 範例：

```
class MyStaticCls
{
    static public int EMP = 1666;
    static public void EEMP()
    static internal string Name { get; set; }
}
```

static的特性

- 不須建立執行個體(new)即可使用。
- 未使用static物件時，仍會占用記憶體空間。
- 在static物件中不能呼叫非static的物件(方法、變數等)。
- 可存放程式中共用的物件。
- 多個物件同時作業，存取會互相影響。

何謂靜態變數

- 靜態變數(欄位)為靜態成員中的一種。
- 必須在方法外宣告。
- 即使方法執行結束，靜態變數的值仍會保留，直到程式終止才會被釋放。
- 不需建立實體物件即可使用，是全域共用的資料成員。
- 範例：
 - `static string ex = "static member" ;`

何謂執行個體變數

- 執行個體變數就是不經**static**宣告的一般變數。
 - `string cry = "哭啊喊啊";`
- 使用**new**建立物件時，此物件稱為執行個體，而賦予這個個體的名稱叫作執行個體變數，可透過變數初始化任何執行個體成員。
 - `MegaPort takao = new MegaPort();`
`takao.Band = "草東沒有派對";`
`takao.Reward = 1000000;`